

SAP-Assignment 3

Shipping on the Air

Marco Morelli

La relazione descrive l'evoluzione del sistema *Shipping on the Air* dall'Assignment 2 all'Assignment 3. Il sistema di partenza è un prototipo di consegna pacchi tramite droni composto da tre microservizi: account-service, delivery-service e api-gateway. L'Assignment 3 interviene su tre assi: ridisegno di un microservizio con architettura event-driven basata su Apache Kafka, definizione di SLO/SLI con deployment Kubernetes, e modellazione del drone secondo un approccio agent-based.

1 EVENT DRIVEN ARCHITECTURE

Viene scelto il delivery-service come microservizio da ridisegnare in chiave event-driven. È il candidato naturale: il ciclo di vita di una consegna è già una sequenza di transizioni di stato modellate come domain event (creazione, validazione, scheduling, riserva e assegnazione del drone, avvio, aggiornamento del tempo stimato, completamento, cancellazione), ereditato dall'Event Sourcing introdotto nell'Assignment 2. In quel contesto convivevano già due piani a eventi, la persistenza a eventi (event store, lato write) e la notifica a eventi (Observer in-process per tracking e metriche).

Kafka aggiunge il terzo piano: la comunicazione a eventi tra servizi. Gli altri due microservizi mantengono l'architettura originale: l'account-service resta invariato (HTTP/REST), il gateway adatta i soli proxy verso il delivery.

Come middleware viene utilizzato **Apache Kafka** a broker singolo, aggiunto come servizio esterno al *docker-compose.yml* insieme a **Zookeeper**. Si adotta una **broker topology** (senza mediatore): gli eventi fluiscono attraverso il broker ai consumatori, secondo il principio del **single writer**: ogni topic ha un unico produttore proprietario (il gateway per i comandi, il delivery per gli eventi di risposta). Questa scelta favorisce l'estensibilità e la tracciabilità della data lineage, senza introdurre un orchestratore centrale che il dominio non richiede.

1.1 Client Kafka e struttura dei canali

L'accesso a Kafka viene incapsulato in due adapter, *OutputEventChannel* (produttore) e *InputEventChannel* (consumatore), presenti sia nel delivery sia nel gateway. Per ogni operazione di dominio viene definito un canale di input (la richiesta) e due canali di output per l'esito (approved/rejected). I canali di ingresso sono **statici**, mentre le risposte legate a una risorsa specifica viaggiano su canali **dinamici**, parametrizzati per *deliveryId*. Le operazioni convertite a **Kafka** sono *create*, *get*, *cancel*, *track* e *stop*.

1.2 Pattern request/reply nel gateway

Il *DeliveryServiceProxy* del gateway riconverte la comunicazione asincrona di **Kafka** in una chiamata sincrona, come richiesto dai controller **REST** lato client. Viene adottata la tecnica del **correlation ID**: per ogni richiesta viene generato un *requestId*, viene registrata una *Promise* in una mappa *requestId* → *Promise*, si assicura la sottoscrizione ai canali di risposta, si posta la richiesta e si attende il completamento della *Promise* con un timeout di 10 secondi. Gli handler dei canali approved/rejected ritrovano la *Promise* tramite il *requestId* e la completano, mappando gli esiti applicativi sui codici *HTTP* corrispondenti. Il timeout della *Promise* realizza il fail-fast lato gateway in caso di indisponibilità del delivery.

1.3 Tracking: dal relay WebSocket al bridge su Kafka

Nell'Assignment 2 il tracking real-time era un relay in due tratti: il gateway accettava la WebSocket dal client e apriva una WebSocket interna verso il delivery. Nell'Assignment 3 quel secondo tratto interno viene eliminato e sostituito da **Kafka**. Il delivery pubblica la telemetria del drone su un topic interno, un forwarder la ripubblica su un topic esterno parametrizzato per *deliveryId*, e il gateway consuma quel topic e lo inoltra al client tramite un bridge *Kafka* → *EventBus* → *WebSocket*. La WebSocket **client↔gateway** resta, perché un browser non consuma un topic Kafka. Tutta la comunicazione tra servizi passa ora da Kafka.

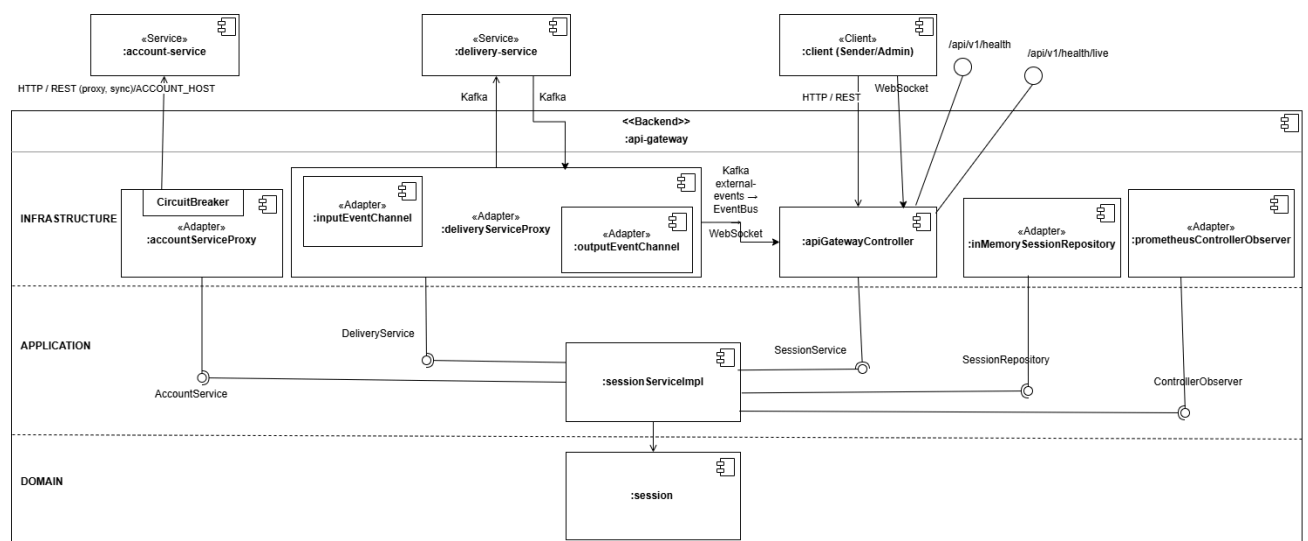
1.4 Confine event-driven: admin e scheduler restano invariati

Il confine event-driven coincide con il flusso dei comandi di dominio della consegna, non con l'intera superficie HTTP del servizio. Le query amministrative (viste su flotta e scheduling) **restano REST**: interrogano read model e non registrano eventi di dominio. Modellarle come stream

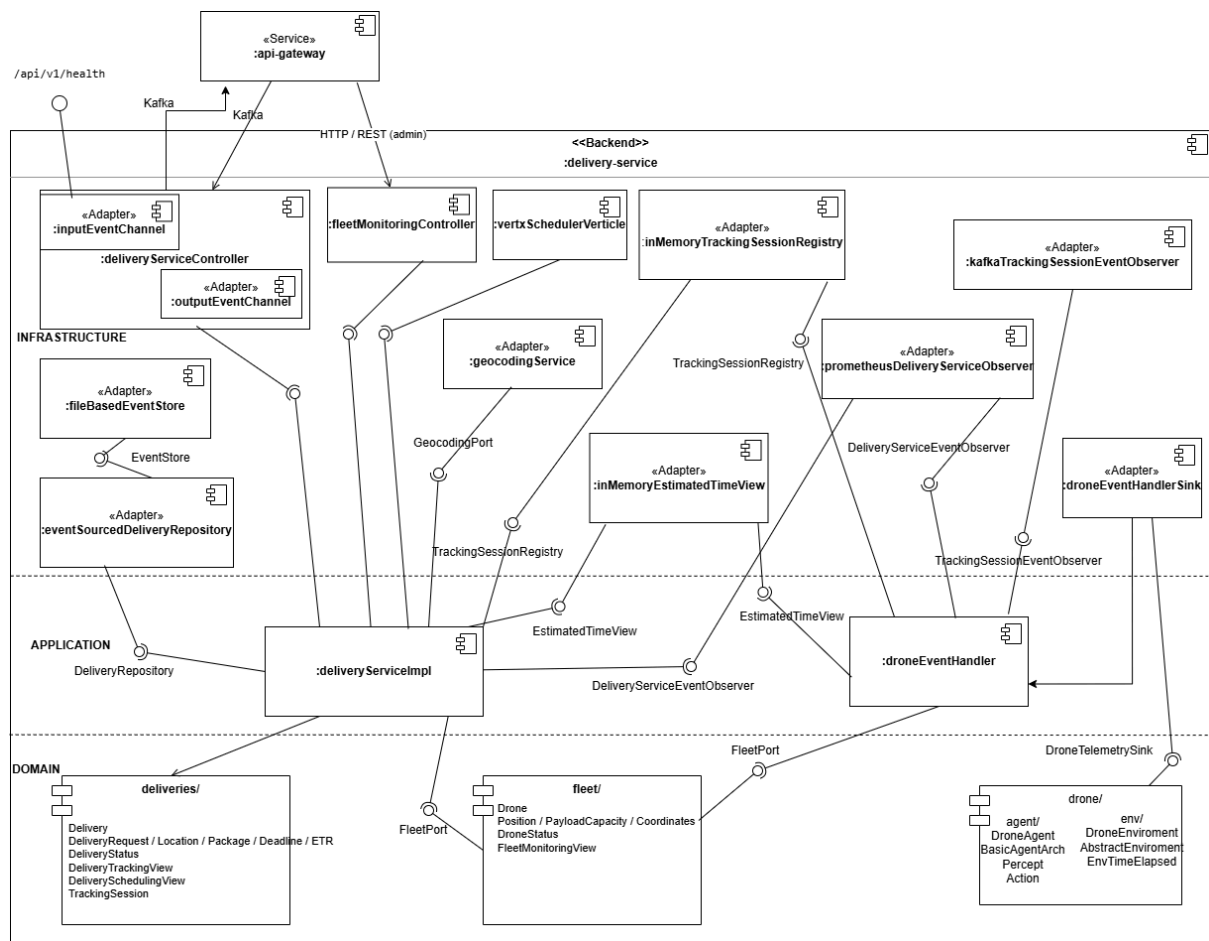
Kafka significherebbe rappresentare come sequenza di fatti immutabili ciò che è una lettura puntuale di stato, snaturando il pattern. Si reintrodurrebbe inoltre la complessità di Kafka senza il beneficio del disaccoppiamento, dato che l'amministratore attende una risposta immediata. Anche lo scheduler delle consegne programmate resta invariato: è interno e non ha alcun impatto su Kafka.

1.5 Circuit Breaker: rimosso dal path Kafka

Nell'Assignment 2 il **Circuit Breaker** era applicato ai proxy del gateway per fronteggiare i partial failure nella comunicazione sincrona. Nell'Assignment 3 viene rimosso dal proxy delivery ma mantenuto sul proxy account, che resta sincrono su *HTTP*. Il breaker fa fail-fast su chiamate sincrone request/response per non accumulare thread bloccati su un downstream irraggiungibile. Con **Kafka** non esiste una chiamata sincrona da interrompere: il produttore posta sul topic e il broker persiste l'evento. Il disaccoppiamento temporale che il breaker emulava, **Kafka** lo fornisce nativamente: se il delivery è indisponibile, l'evento resta nel topic e viene consumato al ripristino, senza richieste perse. Applicare un breaker a un produttore Kafka reintrodurrebbe una protezione contro un blocco sincrono che, per costruzione, non esiste più.



C&C dell'api-gateway



C&C del delivery-service

2 SCALING AND SERVICE LEVELS

2.1 SLO e SLI

Vengono definiti due Service Level Objectives per il sistema:

- **SLO-1 (Availability):** il servizio deve avere una disponibilità superiore al 99% su 30 giorni.
- **SLO-2 (Throughput):** il servizio deve avere un throughput superiore a 40 richieste al secondo su 30 giorni.

E i corrispondenti Service Level Indicators:

- **SLI-1:** percentuale di richieste con successo sul totale.
- **SLI-2:** numero di richieste processate al secondo.

La definizione di availability è **domain-driven**: misura il successo end-to-end attraverso il gateway. Per misurarla *l'observability* del gateway è estesa con il contatore *successful_rest_requests* affiancato a *rest_request*; entrambi escludono per costruzione le rotte di health (/api/v1/health), così

che il traffico delle probe non falsi la misura. Poiché il gateway è deployato con tre repliche, i contatori sono per-replica e devono essere aggregati:

Prometheus scrupa separatamente ciascun pod e gli **SLI** si calcolano via PromQL sommando su tutte le repliche. *Availability* è ricavata come $\text{sum}(\text{successful_rest_requests_total})/\text{sum}(\text{rest_requests_total})$; *throughput* come $\text{sum}(\text{rate}(\text{rest_requests_total}[\dots]))$.

In seguito a queste modifiche sono stati aggiornati i Quality Attribute Scenarios: lo scenario di responsiveness dell'Assignment 2 è sostituito da uno di throughput, viene aggiunto uno scenario di availability, e lo scenario sui partial failure resta invariato.

2.2 Kubernetes

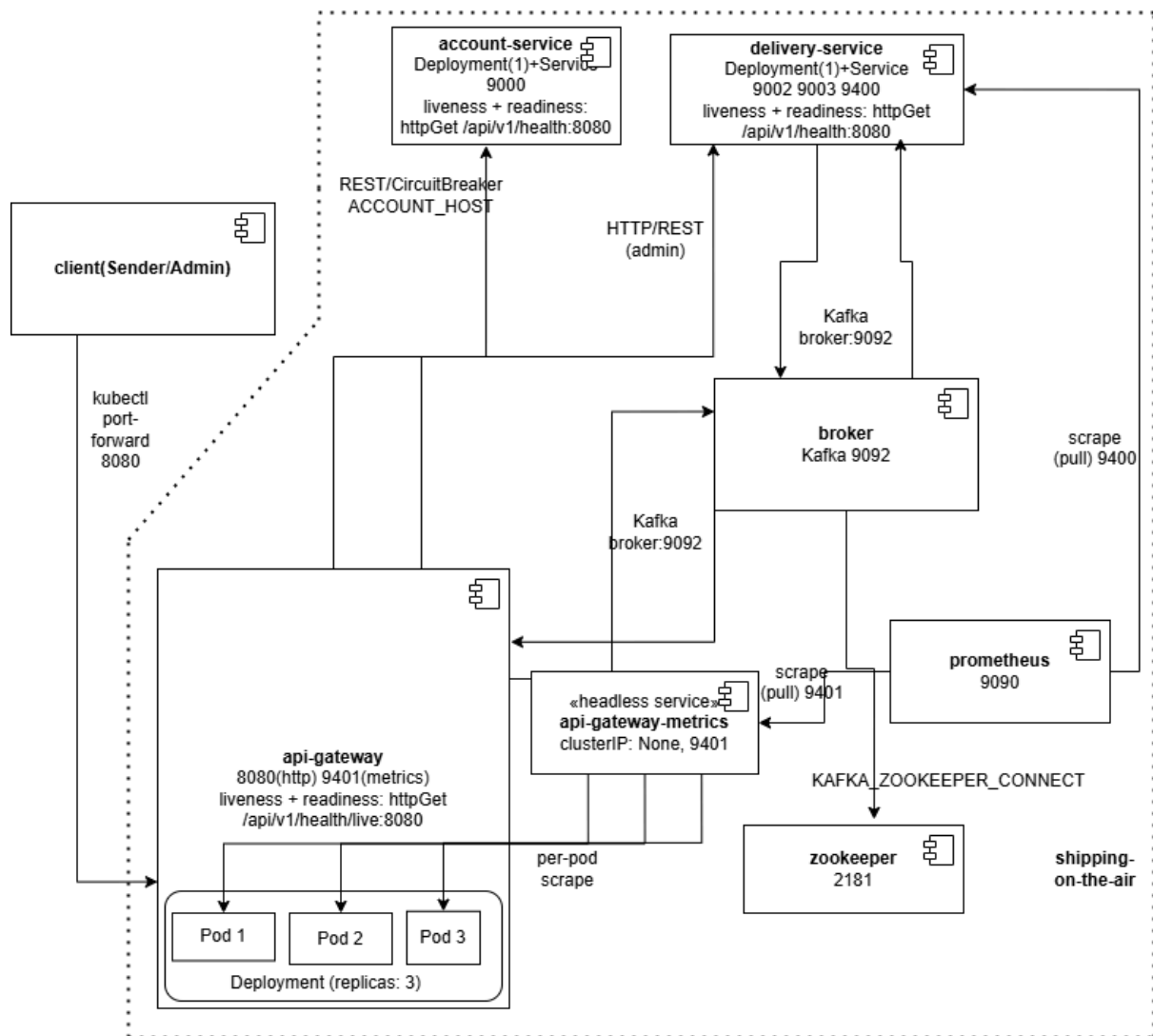
Viene configurato un deployment **Kubernetes** aggiuntivo nel namespace *shipping-on-the-air*, definendo per ogni microservizio un *Deployment* (che gestisce i *Pod* tramite *ReplicaSet*) e un *Service*. L'api-gateway è configurato con **tre repliche**: è stateless e costituisce il collo di bottiglia del sistema, quindi beneficia del load balancing offerto dal *Service*. Gli altri servizi restano a una replica: *account-service* e *delivery-service* possiedono uno stato interno che più repliche frammenterebbero, provocando malfunzionamenti.

La mappa che riconcilia sessione e consegna nel bridge di tracking è in-memory per-istanza; con il gateway a tre repliche, il track e la successiva apertura della *WebSocket* devono raggiungere la stessa replica. Viene usata quindi *sessionAffinity: ClientIP* sul *Service* del gateway: la stessa sorgente resta sulla stessa replica, mentre i comandi REST restano load-balanced. I *Service* sono di tipo *ClusterIP*; l'accesso dall'host avviene tramite *kubectl port-forward*. Le variabili d'ambiente dei manifest rispecchiano quelle del *Compose*, e le immagini sono buildate localmente con *imagePullPolicy: IfNotPresent*.

Coerentemente con la natura di prototipo, la persistenza resta volutamente effimera: non viene montato alcun volume, così lo stato scritto a runtime non sopravvive alla ricreazione del container o del Pod, e i seeder ripristinano uno stato iniziale deterministico a ogni avvio.

Con il gateway a tre repliche, uno scrape verso il *ClusterIP* del *Service* raggiungerebbe una sola replica per intervallo (e, con *sessionAffinity: ClientIP*, sempre la stessa), rendendo gli **SLI** parziali perché calcolati su un

solo pod mentre i comandi **REST** sono distribuiti su tutte e tre. Per misurare l'aggregato reale si introduce un *Service headless* dedicato alle metriche (*clusterIP: None*), il cui DNS risolve gli IP di tutti i pod ready; **Prometheus** lo interroga via *dns_sd_configs*, ottenendo un target per replica. Gli **SLI** si calcolano quindi sommando i contatori delle tre repliche con *sum(...)*. Il *Service* applicativo del gateway resta invariato: l'*headless* serve esclusivamente lo scrape.



C&C di Deployment Kubernetes

3 AGENT BASED ARCHITECTURES

Il drone del delivery-service viene modellato secondo un'architettura ad agente **Sense-Plan-Act**, sostituendo il precedente simulatore basato su un thread ciclico. L'agente vive in un ambiente astratto e alterna tre fasi in un ciclo continuo.

- **Sense:** l'agente percepisce gli eventi dell'ambiente. Nel caso in esame l'unico percept è *EnvTimeElapsed*, il "battito del mondo" emesso ogni secondo da un ambiente condiviso, che sostituisce l'attesa fissa del vecchio simulatore.
- **Plan:** in base allo stato corrente e ai belief, l'agente pianifica l'azione da compiere e aggiorna il proprio stato (macchina a stati *READY_TO_SHIP* → *SHIPPING* → *DELIVERED*). La fase è atomica e non percepisce.
- **Act:** l'agente esegue le azioni pianificate in modo non bloccante. L'avanzamento fa progredire le coordinate geografiche del drone verso la destinazione, aggiornando la posizione e notificando la telemetria; all'arrivo segnala il completamento e si arresta.

Il drone si muove nello spazio e produce telemetria di posizione, che alimenta il tracking real-time già presente nel sistema. Viene adottato il **pattern** Sense-Plan-Act e l'astrazione di ambiente, mantenendo il comportamento geografico del drone e riutilizzando gli aggregate esistenti, così che gli eventi prodotti restino identici.

4 TESTING

La strategia di test conserva parzialmente la struttura a piramide dell'Assignment 2, adattata a **Kafka**. Unit test di dominio e agente, integration test e test architetturali ArchUnit risiedono in ciascun servizio; gli end-to-end in un modulo separato. Con il passaggio dei comandi del delivery a Kafka, il component test black-box via REST del delivery non è più applicabile ed è stato rimosso; il livello component resta sull'account-service.

Gli integration test che dipendono da Kafka adottano un approccio **assumption-based**: iniziano verificando la raggiungibilità del broker e, in sua assenza (esecuzione locale senza broker), vengono saltati anziché falliti. La presenza di un broker reale permette di farli girare su test dedicati nei docker-compose. Gli end-to-end seguono l'approccio user journey (creazione, cancellazione, tracciamento) e includono i test dei Quality Attribute, ovvero il test di throughput, un nuovo test di availability e il test sui partial failure.

5 CONCLUSIONE

L'adozione di un'architettura event-driven basata su **Kafka** ha disaccoppiato le comunicazioni tra gateway e delivery, sostituendo le interazioni sincrone e il relay WebSocket interno con canali a eventi.

La definizione di **SLO** e **SLI** ha formalizzato il livello di reliability desiderato e lo ha reso verificabile tramite **Prometheus**. Il deployment **Kubernetes** ha introdotto scaling orizzontale e load balancing sul gateway stateless, con una gestione consapevole dello stato interno degli altri servizi e dell'affinità di sessione per il tracking.

Infine, la modellazione del drone come agente **Sense-Plan-Act** ha sostituito un comportamento ciclico con una pianificazione guidata dall'ambiente, introducendo l'astrazione di ambiente e preservando al contempo il movimento geografico che alimenta il tracking.